

G Prashanth

192311366

CSA1709-ARTIFICIAL INTELLIGENCE

EXP-17-Write a Prolog Program to Sum the Integers from 1 to n

Aim: To sum the integers from 1 to n

Algorithm:

1. sum(N, Result) :-
2. N > 0, % Ensuring N is a positive integer
3. N1 is N - 1, % Decrementing N by 1
4. sum(N1, SubResult), % Recursive call to sum the integers from 1 to N1
5. :Result is N + SubResult. % Adding N to the sum of integers from 1 to N1 to get the final result

Program:

```
/* Sum of integers from 1 to N */
```

```
sum(0,0).
```

```
sum(N,S):-
```

```
    N>0,
```

```
    N1 is N-1,
```

```
    sum(N1,S1),
```

```
    S is S1+N.
```

Output:

```
| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/sum of n numbers.pl').  
compiling C:/Users/Dharshana/OneDrive/Documents/AI/sum of n numbers.pl for byte code...  
C:/Users/Dharshana/OneDrive/Documents/AI/sum of n numbers.pl compiled, 7 lines read - 864 bytes written, 4 ms
```

```
yes
```

```
| ?- sum(10,S).
```

```
S = 55 ? |
```

Result: The above program was executed and output was verified.

EXP-18-Write a Prolog Program for A DB WITH NAME, DOB.

Aim: to write a program for A DB WITH NAME ,DOB using python

ALGORITHM:

1. Start the program.
2. Define facts containing the person's name and date of birth.
3. Define a predicate dob(Name, Date) to retrieve the DOB.
4. Query the database using a person's name.
5. Display the corresponding DOB.
6. Stop.

Program:

```
% define some facts about people and their DOBs
dob(john, date(1990, 5, 1)).
dob(jane, date(1985, 12, 10)).
dob(bob, date(1978, 2, 28)).
dob(sue, date(1995, 8, 15)).
dob(tom, date(2000, 4, 22)).

% define a predicate to look up a person's DOB by name
lookup(Name, DOB) :-
    dob(Name, DOB).

example query:
lookup john's DOB
?- lookup(john, DOB).
DOB = date(1990, 5, 1).

% example query:
lookup sue's DOB
?- lookup(sue, DOB).
DOB = date(1995, 8, 15).
```

OUTPUT:

```
| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/dob with name.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/dob with name.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/dob with name.pl compiled, 6 lines read - 1215 bytes written, 6 ms

yes
| ?- lookup(john,DOB).

DOB = date(1990,5,1)

yes
```

Result: Hence the Prolog Program for A DB WITH NAME, DOB is done.

EXP-19- Write a Prolog Program for STUDENT-TEACHER-SUB CODE.

Aim: To find the student teacher code

ALGORITHM:

1. Start the program.
2. Define facts for students and their subjects.
3. Define facts for teachers and the subjects they handle.
4. Define subject codes.
5. Create a predicate to relate student, teacher, and subject code.
6. Query the database.
7. Display the result.
8. Stop.

Program:

```
% define some facts about teachers and their subject codes

teaches(john, math).

teaches(jane, english).

teaches(bob, science).

teaches(sue, history).
```

```

teaches(tom, art).

% define some facts about students and the subjects they are taking
takes(alice, math).
takes(alice, science).
takes(bob, english).
takes(bob, science).
takes(carol, history).
takes(carol, art).
takes(dave, math).
takes(dave, english).
takes(dave, art).

% define a predicate to look up the subject codes taught by a teacher
teaching_subjects(Teacher, Subject) :-
    teaches(Teacher, Subject)

% define a predicate to look up the students taking a given subject
taking_students(Subject, Student) :-
    takes(Student, Subject).

% example query: find the subjects taught by john
?- teaching_subjects(john, Subject).

Subject = math.

% example query: find the students taking science
?- taking_students(science, Student).

Student = alice .

% example query: find the students taking math and art
?- taking_students(math, Student), taking_students(art, Student).

; Student = dave.

```

OUTPUT:

```
| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/STUDENT-TEACHER-SUB.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/STUDENT-TEACHER-SUB.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/STUDENT-TEACHER-SUB.pl compiled, 17 lines read - 1934 bytes written, 6 ms

(15 ms) yes
| ?- teaching_subjects(john, Subject).

Subject = math

yes
| ?- taking_students(science, Student).

Student = alice ?

yes
| ?- taking_students(math, Student), taking_students(art, Student).

Student = dave ? |
```

Result: Hence Prolog Program for STUDENT-TEACHER-SUB-CODE is done.

EXP-20- Write a Prolog Program for PLANETS DB.

Aim: To sum the integers from 1 to n

ALGORITHM:

1. Define facts containing the names of planets.
2. Store information such as order, type, and number of moons.
3. Define predicates to retrieve planet information.
4. Query the database.
5. Display the result.
6. Stop.

Program:

planet(mercury, 1, terrestrial, 0).

planet(venus, 2, terrestrial, 0).

```

planet(earth, 3, terrestrial, 1).
planet(mars, 4, terrestrial, 2).
planet(jupiter, 5, gas_giant, 95).
planet(saturn, 6, gas_giant, 146).
planet(uranus, 7, ice_giant, 28).
planet(neptune, 8, ice_giant, 16).
planet_info(Name, Order, Type, Moons) :-
    planet(Name, Order, Type, Moons).

```

Output:

```

| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/PLANETS DB.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/PLANETS DB.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/PLANETS DB.pl compiled, 9 lines read - 1591 bytes written, 3 ms

yes
| ?- planet_info(earth,Order,Type,Moons).

Moons = 1
Order = 3
Type = terrestrial

yes

```

Result: Hence Prolog Program for PLANETS DB is done.

21. Write a Prolog Program to Implement Towers of Hanoi

Aim

To write and execute a Prolog program to solve the Towers of Hanoi problem.

Algorithm

1. Start with all the disks placed on the source peg.
2. If there is only one disk, move it directly to the destination peg.
3. Otherwise, move the top $N-1$ disks from the source peg to the auxiliary peg.

4. Move the remaining largest disk from the source peg to the destination peg.
5. Move the $N-1$ disks from the auxiliary peg to the destination peg.
6. Repeat the process until all the disks are moved successfully.

Program

move(1, Source, Destination, _) :-

```
    write('Move the top disk from '),  
    write(Source),  
    write(' to '),  
    write(Destination),  
    nl.
```

move(N, Source, Destination, Auxiliary) :-

```
    N > 1,  
    M is N - 1,  
    move(M, Source, Auxiliary, Destination),  
    move(1, Source, Destination, Auxiliary),  
    move(M, Auxiliary, Destination, Source).
```

Sample Output

?- move(3,left,right,center).

Move the top disk from left to right

Move the top disk from left to center

Move the top disk from right to center

Move the top disk from left to right

Move the top disk from center to left

Move the top disk from center to right

Move the top disk from left to right

true.

```
| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/Towers of hanoi.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/Towers of hanoi.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/Towers of hanoi.pl compiled, 12 lines read - 1457 bytes written, 5 ms

yes
| ?- move(3,left,right,center).
Move the top disk from left to right
Move the top disk from left to center
Move the top disk from right to center
Move the top disk from left to right
Move the top disk from center to left
Move the top disk from center to right
Move the top disk from left to right

true ?
```

Result

Thus, the Prolog program to implement the Towers of Hanoi problem was executed successfully.

22. Write a Prolog Program to Print Particular Bird Can Fly or Not. Incorporate Required Queries

Aim

To write and execute a Prolog program to determine whether a particular bird can fly.

Algorithm

1. Define the birds and their flying properties.
2. Create a rule to identify birds that can fly.
3. Execute queries for different birds.
4. Display whether the bird can fly or not.
5. List all birds that cannot fly when required.

Program

```
% define some facts about birds and their properties

bird(ostrich, cannot_fly).

bird(penguin, cannot_fly).

bird(kiwi, cannot_fly).

bird(eagle, can_fly).
```



```
bird(pigeon, can_fly).
```

```
bird(sparrow, can_fly).
```

```
% define a predicate to check if a bird can fly
```

```
can_fly(Bird) :-
```

```
    bird(Bird, can_fly).
```

```
% example queries
```

```
?- can_fly(eagle).
```

```
?- can_fly(ostrich).
```

```
?- bird(Bird, cannot_fly).
```

Sample Output

```
?- can_fly(eagle).
```

```
true.
```

```
?- can_fly(ostrich).
```

```
false.
```

```
?- bird(Bird, cannot_fly).
```

```
Bird = ostrich ;
```

```
Bird = penguin ;
```

```
Bird = kiwi.
```

```
| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/bird can fly or not.pl').  
compiling C:/Users/Dharshana/OneDrive/Documents/AI/bird can fly or not.pl for byte code...  
C:/Users/Dharshana/OneDrive/Documents/AI/bird can fly or not.pl:21: warning: singleton variables [Bird] for (?-)/1  
C:/Users/Dharshana/OneDrive/Documents/AI/bird can fly or not.pl compiled, 20 lines read - 1379 bytes written, 4 ms  
  
(16 ms) yes  
| ?- can_fly(eagle).  
  
yes  
| ?-
```

Result

Thus, the Prolog program to determine whether a particular bird can fly or not was implemented and executed successfully.

23. Write the Prolog Program to Implement Family Tree

Aim

To write and execute a Prolog program to represent and query family relationships.

Algorithm

1. Define the male and female members of the family.
2. Define the parent-child relationships.
3. Create rules for mother, father, grandmother, grandfather, sister, and brother.
4. Execute queries to identify family relationships.
5. Display the obtained results.

Program

```
% Facts about the family members
```

```
female(pam).
```

```
female(liz).
```

```
female(ann).
```

```
female(pat).
```

```
male(tom).
```

```
male(bob).
```

```
male(jim).
```

```
% Parent relations
```

```
parent(tom, liz).
```

```
parent(tom, bob).
```

```
parent(pam, liz).
```

```
parent(pam, bob).
```

```
parent(bob, ann).
```

```
parent(bob, pat).
```

```
parent(liz, jim).
```

% Mother relation

mother(X, Y) :-

female(X),

parent(X, Y).

% Father relation

father(X, Y) :-

male(X),

parent(X, Y).

% Grandmother relation

grandmother(X, Y) :-

female(X),

parent(X, Z),

parent(Z, Y)

% Grandfather relation

grandfather(X, Y) :-

male(X),

parent(X, Z),

parent(Z, Y).

% Sister relation

sister(X, Y) :-

female(X),

parent(Z, X),

parent(Z, Y),

$X \neq Y$.

% Brother relation

brother(X, Y) :-

male(X),

parent(Z, X),

parent(Z, Y),

X \= Y.

Sample Queries

?- father(tom, liz).

?- mother(pam, bob).

?- grandfather(tom, ann).

?- grandmother(pam, ann).

?- sister(liz, bob).

?- brother(bob, liz).

Sample Output

?- father(tom, liz).

true.

?- mother(pam, bob).

true.

?- grandfather(tom, ann).

true.

?- grandmother(pam, ann).

true.

?- sister(liz, bob).

true.

?- brother(bob, liz).

true.

```
| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/family tree.pl').  
compiling C:/Users/Dharshana/OneDrive/Documents/AI/family tree.pl for byte code...  
C:/Users/Dharshana/OneDrive/Documents/AI/family tree.pl compiled, 61 lines read - 3763 bytes written, 5 ms
```

```
yes  
| ?- grandfather(tom, ann).
```

```
true ? f
```

Result

Thus, the Prolog program to implement a family tree and identify various family relationships was implemented and executed successfully.

24. Write a Prolog Program to Suggest Dieting System Based on Disease

Aim

To write and execute a Prolog program to suggest a suitable diet based on a given disease.

Algorithm

1. Define different food items and their properties.
2. Store the recommended diet for each disease.
3. Accept the disease name as input.
4. Retrieve the corresponding diet from the knowledge base.
5. Display the recommended diet for the given disease.

Program

```
% Define the foods and their properties
```

```
food(apple, fruit, sweet, low_calorie).
```

```
food(banana, fruit, sweet, high_calorie).
```

```
food(carrot, vegetable, savory, low_calorie).
```

```
food(potato, vegetable, savory, high_calorie).
```

```
food(chicken, meat, savory, high_protein).
```

```
food(fish, seafood, savory, high_protein).
```

```
food(spinach, vegetable, savory, high_iron).
```

```
food(almonds, nut, savory, high_fat).
```

```
% Define the recommended diet based on the disease
```

```
diet(heart_disease, [apple, carrot, chicken, fish, almonds]).
```

```
diet(diabetes, [apple, carrot, fish, spinach, almonds]).
```

```
diet(anemia, [spinach, chicken, fish, almonds]).
```

```
diet(obesity, [apple, carrot, fish, spinach]).
```

```
% Define the rules to suggest the diet based on the disease
```

```
suggest_diet(Disease, Diet) :-
```

```
    diet(Disease, Diet).
```

```
suggest_diet(Disease, Diet) :-
```

```
    diet(Disease, AllowedFoods),
```

```
    findall(Food,
```

```
        (food(Food, _, _, _),
```

```
        member(Food, AllowedFoods)),
```

```
    Diet).
```

```
% Sample Queries
```

```
% ?- suggest_diet(heart_disease, Diet).
```

```
% ?- suggest_diet(diabetes, Diet).
```

```
% ?- suggest_diet(anemia, Diet).
```

```
% ?- suggest_diet(obesity, Diet).
```

Sample Output

```
?- suggest_diet(heart_disease, Diet).
```

```
Diet = [apple, carrot, chicken, fish, almonds].
```

```
?- suggest_diet(diabetes, Diet).
```

```
Diet = [apple, carrot, fish, spinach, almonds].
```

```
?- suggest_diet(anemia, Diet).
```

```
Diet = [spinach, chicken, fish, almonds].
```

```
?- suggest_diet(obesity, Diet).
```

```
Diet = [apple, carrot, fish, spinach].
```

```

| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/dieting system.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/dieting system.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/dieting system.pl compiled, 35 lines read - 3489 bytes written, 4 ms

(15 ms) yes
| ?- suggest_diet(diabetes, Diet).

Diet = [apple,carrot,fish,spinach,almonds] ?

yes
| ?- suggest_diet(obesity, Diet).

Diet = [apple,carrot,fish,spinach] ?

```

Result

Thus, the Prolog program to suggest a dieting system based on disease was implemented and executed successfully.

25. Write a Prolog Program to Implement Monkey Banana Problem

Aim

To write and execute a Prolog program to solve the Monkey Banana problem using state-space representation.

Algorithm

1. Define the initial state of the monkey.
2. Define the goal (final) state.
3. Specify the possible actions and their effects.
4. Execute the sequence of actions from the initial state.
5. Continue until the monkey reaches the goal state.
6. Display the sequence of actions.

Program

```

% Initial state

initial_state(state(at_door,on_floor,at_window,has_not_eaten)).

% Goal state

final_state(state(_,_,_,has_eaten)).

% Actions

action(state(at_door,on_floor,at_window,has_not_eaten),

```

```

    walk,

    state(at_window,on_floor,at_window,has_not_eaten)).
action(state(at_window,on_floor,at_window,has_not_eaten),

    climb,

    state(at_window,on_box,at_window,has_not_eaten)).
action(state(at_window,on_box,at_window,has_not_eaten),

    grasp,

    state(at_window,on_box,at_window,has_eaten)).

```

```

% Execute actions

execute(State, [], State).

execute(State, [Action|Rest], FinalState) :-
    action(State, Action, NextState),
    execute(NextState, Rest, FinalState).

```

```

% Solve problem

solve_problem(ActionList) :-
    initial_state(InitialState),
    execute(InitialState, ActionList, FinalState),
    final_state(FinalState).

```

Sample Output

?- solve_problem(ActionList).

ActionList = [climb, walk, grasp].

```

| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/monkey banana.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/monkey banana.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/monkey banana.pl compiled, 33 lines read - 2968 bytes written, 4 ms

(15 ms) yes
| ?- solve_problem(ActionList).
ActionList = [walk,climb,grasp] ?

```


Result

Thus, the Prolog program to solve the Monkey Banana problem was implemented and executed successfully.

26. Write a Prolog Program for Fruit and Its Color Using Backtracking

Aim

To write and execute a Prolog program to identify fruits and their colors using backtracking.

Algorithm

1. Define fruits and their respective colors.
2. Create a predicate to match fruits with their colors.
3. Use backtracking to retrieve all fruits of a specified color.
4. Execute the required queries.
5. Display the matching fruits and colors.

Program

```
% Define the possible fruits and their colors
```

```
fruit(apple, red).
```

```
fruit(banana, yellow).
```

```
fruit(grape, purple).
```

```
fruit(orange, orange).
```

```
fruit(watermelon, green).
```

```
% Define a predicate to match a fruit with its color
```

```
match_fruit_color(Fruit, Color) :-
```

```
    fruit(Fruit, Color).
```

```
% Define a predicate to find all fruits with a certain color
```

```
% Note: The color argument is expressed as a variable to enable backtracking
```

```
fruits_with_color(FruitList, Color) :-  
    findall(Fruit,  
        match_fruit_color(Fruit, Color),  
        FruitList).
```

% Sample Queries

```
% ?- match_fruit_color(apple, Color).  
% ?- match_fruit_color(banana, Color).  
% ?- match_fruit_color(pear, Color).  
% ?- fruits_with_color(FruitList, red).  
% ?- fruits_with_color(FruitList, green).  
% ?- fruits_with_color(FruitList, purple).
```

Sample Output

```
?- match_fruit_color(apple, Color).  
Color = red.  
?- match_fruit_color(banana, Color).  
Color = yellow.  
?- match_fruit_color(pear, Color).  
false.  
?- fruits_with_color(FruitList, red).  
FruitList = [apple].  
?- fruits_with_color(FruitList, green).  
FruitList = [watermelon].  
?- fruits_with_color(FruitList, purple).  
FruitList = [grape].
```

```
| | ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/fruit colour using backtracking.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/fruit colour using backtracking.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/fruit colour using backtracking.pl compiled, 28 lines read - 1205 bytes written, 4 :
yes
| | ?- match_fruit_color(apple, Color).
Color = red
yes
| | ?- fruits_with_color(FruitList, purple).
FruitList = [grape]
yes .
```

Result

Thus, the Prolog program for identifying fruits and their colors using backtracking was implemented and executed successfully.